

1ª Prova de Linguagens de Programação

Aluno _____

Data 26/07/2024

Prof. Ricardo Couto A. da Rocha

1ª Questão (2,0 pontos)

Considere o problema de implementar uma função `split` que recebe uma string e retorna uma string com os caracteres da string original com o caracter “,” (e espaço) delimitando sempre que houver uma mudança no caracter na string. Assim, a invocação

```
split("gHHH5YY++///x")
```

deveria retornar a string

```
"g, HHH, 5, YY, ++, ///, x"
```

Considere agora as duas implementações de `split` em C e em Python abaixo.

```
char *split(char *str)
{
    char ultimo = *str;
    char *resultado = malloc(3*strlen(str));
    char *contador = resultado;
    for (char *pos_c = str; *pos_c != '\0'; pos_c++) {
        if (*pos_c != ultimo) {
            strcpy(contador, ", ");
            contador = contador + 2;
            ultimo = *pos_c;
        }
        *contador = *pos_c;
        contador++;
    }
    *(contador--) = '\0';
    return realloc(resultado, strlen(resultado) );
}
```

Para a implementação em C:

- A função `strlen` retorna o tamanho de uma string terminada em nulo
- A função `realloc` muda o tamanho do espaço em memória ocupado por um vetor para o tamanho indicado, fazendo uma alocação em uma nova área de memória se necessário e copiando o conteúdo do vetor para a nova área. O ponteiro retornado pode ser no mesmo endereço ou não daquele passado como parâmetro, dependendo se uma nova alocação ocorre.

Implementação em Python:

```
def split(str):
    ultimo = str[0]
    resultado = [ultimo]
    for pos_c in range(1, len(str)):
        if str[pos_c] != ultimo:
            resultado = resultado + [", "]
            ultimo = str[pos_c]

        resultado = resultado + [str[pos_c]]

    return ''.join(resultado)
```

1. Compare a legibilidade, redigibilidade e confiabilidade em C e em Python a partir das características das linguagens ilustradas pelos códigos.
2. Descreva a amarração de armazenagem de todas as variáveis nos dois programas, indicando se é estática ou dinâmica e onde ocorrerá a armazenagem (memória).

- str
- ultimo
- resultado
- contador
- pos_c

2ª Questão (2,0 pontos)

Considere o trecho de código abaixo em uma linguagem hipotética.

```
LIM = 2
i = 0
j = 0

def f3():
    print("f3:i=", i, " j=", j)

def f1(p):
    def f3():
        print("f3/f1:i=", i, " j=", j+2)

    i = j + p
    print("f1:i=", i, " j=", j)

    if (i < LIM):
        f2(i)

def f2(p):
    j = i + p
    print("f2:i=", i, " j=", j)
    if (j < LIM):
        f3()

while (i < LIM):
    while (j < LIM):
        f1(i+1)
        j = j + 1
    i = i + 1
```

FL 1 0
F2 1 2
FL 2 1
FL 2 0
FL 3 1

1. Qual será a saída do programa (produzida pelos comandos `print`), caso a linguagem utilize **ESCOPO ESTÁTICO**?
2. Qual será a saída do programa (produzida pelos comandos `print`), caso a linguagem utilize **ESCOPO DINÂMICO**?

3ª Questão (2,0 pontos)

Considere a seguinte declaração de uma matriz `m` de 3x3x3:

```
int m[3][3][3];
```

Desenhe dois diagramas mostrando a disposição em memória dos elementos da matriz em memória de acordo com as seguintes linguagens. Mostre em ambos os diagramas a posição do elemento `m[0][0][0]` e `m[1][1][0]`.

- (a) C
- (b) Java (matriz implementada como vetor de vetores).

4ª Questão (2,0 pontos)

Para o código C abaixo:

```
typedef struct L {
    char *id;
    struct L *next;
} L;

void f(L *l_parametro_1, L *l_parametro_2) {
    L l1, l2, l3;
    l1.id = "obj1";
    l2.id = "obj2";
    l3.id = "obj3";
    l1.next = &l2;
    l2.next = &l3;
    l3.next = &l2;
    l_parametro_1->next = &l1;

    L *l4, *l5, *l6;
    l4 = (L *) malloc(sizeof(L));
    l5 = (L *) malloc(sizeof(L));
    l6 = (L *) malloc(sizeof(L));
    l4->id = "obj4";
    l5->id = "obj5";
    l6->id = "obj6";
    l4->next = l5;
    l5->next = l6;
    l6->next = l5;
    l_parametro_2->next = l4;
}
```



Considere uma invocação `f(primeira_ref, segunda_ref)`.

1. Quais variáveis serão armazenadas na pilha e quais serão armazenadas no heap?
2. A variáveis `primeira_ref` e `segunda_ref` mantêm referências consistentes e válidas ao fim da invocação? Justifique.
3. Quais desalocações de memória (`free(...)`) devem ser feitas ao fim de `f()` que garantem que o código poderá continuar a execução sem problemas no acesso à memória heap. Justifique. **Análise com cuidado!**

5ª Questão (2,0 pontos): Assinale a alternativa CORRETA em cada uma das questões objetivas abaixo.

Considere a implementação em C abaixo da função `soma_vetores` que soma os elementos de dois vetores

```
int *soma_vetores(int vet1[], int vet2[], int dim) {
    int v_soma[dim];
    int i = 0;
    for (i = 0; i < dim; i++) {
        v_soma[i] = vet1[i] + vet2[i];
    }
    return v_soma;
}
```

5.1. Em qual área da memória feita a amarração de armazenagem dos vetores referenciados por `vet1`, `vet2` e `v_soma`, respectivamente?

- Pilha*
- a) Depende do código que invoca `soma_vetores` para todos os vetores. ✗
 - ☒ b) Depende do código que invoca `soma_vetores`, para `vet1`, `vet2`. Para `v_soma` ocorre na pilha.
 - c) Ocorre na pilha para todos os vetores.
 - d) Ocorre no heap para todos os vetores. ✗

5.2. Uma função implementada em uma linguagem sem gerenciamento automático de memória, como C, precisa retornar o endereço de uma variável (por exemplo, um vetor). Em qual das situações abaixo, o endereço pode ser retornado com segurança e sob qual(is) condição(ões)?

- a) Variável é de PILHA e não é necessária nenhuma preocupação adicional.
- b) Variável é de PILHA e desalocação da memória fica a cargo da função invocada.
- c) Variável é de PILHA e desalocação da memória fica a cargo do código usando o valor de retorno.
- d) Variável é de PILHA e desalocação da memória ocorre ao fim do escopo da variável.
- e) Variável é de HEAP e não é necessária nenhuma preocupação adicional.
- ☒ f) Variável é de HEAP e desalocação da memória fica a cargo do código usando o valor de retorno.
- g) Variável é de HEAP e desalocação da memória fica a cargo da função invocada.
- h) Variável é de HEAP e desalocação da memória ocorre ao fim do escopo da variável.

5.3. Em uma linguagem com sistema de gerenciamento automático de memória (coleta de lixo), quando há referências cíclicas (circulares) entre variáveis

- a) As variáveis não devem ser desalocadas.
- ☒ b) A contagem de referências não é capaz de detectar quando os blocos de memória usados pelas variáveis poderão ser desalocados.
- c) O algoritmo marcar-e-varrer (mark-and-sweep) se encarrega de garantir que o número de referências para cada bloco de memória, não ultrapassa 1 (um).
- d) A desalocação de memória pelo coletor de lixo pode entrar em loop.